Harness Engineering: A Code-First Guide to Production LLM Agents

A code-first synthesis of Codex, Goose, OpenCode, OpenClaw, NanoClaw, and Hermes

Generated 2026-06-27

Table Of Contents

Preface

This guide is intentionally code-first. It extracts recurring production patterns from code paths that own sessions, turns, tools, context, permissions, delivery, testing, and operations. Appendix B is the audit trail: each major claim is tied to local source paths and short line-anchored excerpts.

The central argument is simple: production LLM agents are not model calls with helpers. They are harnesses that admit work, build context, expose capabilities, gate authority, recover from failures, publish product state, and leave evidence.

Chapter 1: What A Coding Harness Is

Thesis: A coding harness is the production runtime around a model: it admits work, renders context, exposes tools, gates authority, records side effects, and makes the run legible.

The six repositories make the same architectural argument in different languages. Codex centers the Rust turn/session engine. Goose wraps MCP-shaped tools and desktop/server surfaces around a Rust agent. OpenCode separates prompt admission from V2 execution in a Bun/Effect runtime. OpenClaw turns providers, plugins, channels, policies, and native clients into a wide platform. NanoClaw makes host/container isolation and SQLite message files the boundary. Hermes keeps a Python agent facade but extracts prompt, tools, approval, gateway, browser, desktop, and observability concerns into owned subsystems.

The harness is therefore not decoration around a model call. It is the system of record for what the user asked, what the model saw, which tools were visible, why a command was allowed, what side effects were requested, whether a result was delivered, and what can be replayed after a crash. A demo can keep those facts in local variables. A production agent has to store and expose them.

This distinction changes design review. Instead of asking whether the agent can call a shell, ask where shell authority is represented as data. Instead of asking whether the model has memory, ask who owns each memory source, when it is loaded, and how it is bounded. Instead of asking whether the UI looks helpful, ask whether it reflects the runtime's true states: sampling, waiting for approval, running a tool, compacting, interrupted, blocked, delivered.

Practices

- Treat the model as a dependency behind the harness, not as the architecture.
- Name durable runtime facts first: session, run, turn, step, tool call, approval, delivery, usage, and error.
- Make every high-authority boundary visible in code before exposing it through prompts.

Code Walk-Through

- E01: A production agent is a session coordinator, not just a while-loop around tool calls. Sources: codex/codex-rs/core/src/session/turn.rs:128; goose/crates/goose/src/agents/agent.rs:1888
- E15: A smaller core with a wider edge is the dominant shape. Sources: codex/codex-rs/core/src/session/session.rs:51; goose/crates/goose/src/agents/agent.rs:236

Representative excerpt: codex/codex-rs/core/src/session/turn.rs:128

```
128: /// Takes initial turn input and runs a loop where, at each sampling request, 129: /// the model replies with either:
```

Builder Transfer

- Own prompt admission, active-turn serialization, continuation, cancellation, and finalization as explicit state transitions.
- Keep model/provider/tool/plugin/channel code behind narrow runtime contracts so new surfaces do not rewrite the loop.

Chapter 2: The Common Architecture Of Production Autonomous Agents

Thesis: The common shape is a small active-run coordinator surrounded by provider, context, tool, policy, persistence, delivery, and observability systems.

Across the repos, the production loop is a coordinator. Codex `run_turn` compacts, captures world state, injects skills/plugins, builds tools, samples, dispatches, records, and follows up. OpenCode promotes queued input, resolves context/model/tools, streams one provider turn, settles tools, and loops only for explicit continuations. Goose streams provider events, classifies tools, inspects permissions, asks for approval, dispatches tool streams, and persists responses. OpenClaw wraps the loop in auth profiles, queue lanes, policy layers, context engines, provider fallback, channel delivery, and cleanup. NanoClaw routes messages through a host and one active container. Hermes persists turns before side effects and runs tool calls through a controlled executor.

That architecture has a core invariant: the harness owns state transitions, not the model. The model may propose a tool call, but the harness decides whether the call is valid, stale, allowed, executed, retried, truncated, or surfaced to the user. The model may produce final text, but the harness decides whether it is addressed to the right destination and whether delivery succeeded.

The reusable pattern is a hexagon: user surfaces and channels at the outside; session store, context builder, provider adapter, tool registry, policy engine, and event bus in the middle; host capabilities and secrets below the model line. This shape lets the same agent support a CLI, TUI, desktop app, API server, messaging channel, or scheduler without rewriting the loop.

Practices

- Separate prompt admission from execution so queued input, steering, cancellation, and replay have a single gate.
- Make one provider turn a bounded operation with explicit setup, stream handling, settlement, and continuation rules.
- Publish runtime state as typed events that every product surface can consume.

Code Walk-Through

- E01: A production agent is a session coordinator, not just a while-loop around tool calls. Sources: codex/codex-rs/core/src/session/turn.rs:128; goose/crates/goose/src/agents/agent.rs:1888
- E02: Durability starts before side effects. Sources: opencode/packages/core/src/session/runner/llm.ts:66; hermes-agent/agent/conversation_loop.py:4203
- E08: Product UX is part of the harness. Sources: codex/codex-rs/tui/src/main.rs:50; goose/ui/desktop/src/App.tsx:598

Representative excerpt: codex/codex-rs/core/src/session/turn.rs:128

```
128: /// Takes initial turn input and runs a loop where, at each sampling request, 129: /// the model replies with either:
```

Builder Transfer

- Own prompt admission, active-turn serialization, continuation, cancellation, and finalization as explicit state transitions.

- Record tool requests or assistant tool-call turns before executing tools so crash recovery and replay have ground truth.
- Stream reasoning, tool metadata, approvals, partial replies, delivery evidence, snapshots, and resumable state to users and clients.

Chapter 3: Repository Organization Patterns

Thesis: The mature repositories organize by runtime boundary: core loop, provider adapters, tool implementations, permission policy, context, UI surfaces, tests, and ops.

Language choices differ, but boundaries repeat. Codex keeps protocol types, core session logic, tools, sandboxing, TUI, app-server, MCP, cloud, and SDKs in distinct workspaces. Goose separates Rust crates for agent/core/MCP/server flows from Electron/React and text UI packages. OpenCode uses a TypeScript monorepo with core services, opencode runtime, app, TUI, plugin, protocol, and docs packages. OpenClaw is broader still: agents, gateway, plugin SDK, native clients, channels, infra, security, context, and tests. NanoClaw splits host daemon/CLI, per-session container runner, skills, setup, and channel adapters. Hermes separates agent core, tools, gateway, apps, skills, plugins, and tests.

The organization teaches ownership. Tool schemas should not live only in prompt files. Prompt renderers should not hide filesystem policy. Browser control should not sit in the same bucket as pure text formatting. Observability should not be an afterthought in UI code. The strongest repos make these boundaries discoverable by filesystem layout and package naming.

A useful repo inventory for a new harness should list app surfaces, model/provider adapters, tool systems, approval surfaces, persistence stores, test harnesses, deployment or service scripts, and telemetry paths. If any of those are hidden in a single large agent file, the repo is still in prototype shape.

Practices

- Organize modules by authority and lifecycle, not by implementation convenience.
- Keep generated protocol/schema packages separate from human-authored runtime policy.
- Make ops and test entrypoints obvious enough that a new engineer can reproduce a production incident locally.

Code Walk-Through

- E15: A smaller core with a wider edge is the dominant shape. Sources: codex/codex-rs/core/src/session/session.rs:51; goose/crates/goose/src/agents/agent.rs:236

Representative excerpt: codex/codex-rs/core/src/session/session.rs:51

```
51: #[derive(Clone)] 52: pub(crate) struct SessionConfiguration {
```

Builder Transfer

- Keep model/provider/tool/plugin/channel code behind narrow runtime contracts so new surfaces do not rewrite the loop.

Chapter 4: Agent Loops, Planning, Execution, And Reflection

Thesis: Planning and reflection are useful only when the loop can persist, interrupt, compact, and recover each execution phase.

The codebases do not treat planning as a magic prompt. They treat it as work admitted into a runtime. A plan can be an explicit mode, a system prompt fragment, a tool result, a subagent task, or a user-visible checklist, but execution still goes through the same settlement machinery. Codex and OpenCode demonstrate why: compaction, follow-up turns, and tool results can change what the next step should be. Goose and Hermes show how approval and tool inspection interrupt the stream before side effects. OpenClaw and NanoClaw show how channel delivery and queued messages can steer or constrain the run.

Reflection has the same constraint. A model can critique its own work only if the harness gives it reliable facts: tool result status, command output budgets, delivery receipts, context summaries, and errors that are classified rather than thrown away. Without those facts, reflection is just a second guess over a partial transcript.

The practical loop is: durably admit input, render context, materialize allowed tools, make one provider call, persist tool intent before side effects, authorize and execute tools, persist typed results, decide continuation, and publish state. Planning, review, and reflection fit into that loop as explicit transitions, not side channels.

Practices

- Define allowed continuation reasons: tool results, user steering, queued work, compaction, retry, or explicit handoff.
- Record planning artifacts and checklist state with the same durability as other assistant outputs.
- Return model-correctable errors to the model only when the model has enough context to repair them.

Code Walk-Through

- E01: A production agent is a session coordinator, not just a while-loop around tool calls. Sources: codex/codex-rs/core/src/session/turn.rs:128; goose/crates/goose/src/agents/agent.rs:1888
- E02: Durability starts before side effects. Sources: opencode/packages/core/src/session/runner/llm.ts:66; hermes-agent/agent/conversation_loop.py:4203
- E07: Compaction is a control path, not a background cleanup task. Sources: codex/codex-rs/core/src/session/turn.rs:345; goose/crates/goose/src/context_mgmt/mod.rs:1
- E12: Repeated tool-call and malformed-tool recovery should be first-class. Sources: hermes-agent/agent/conversation_loop.py:3972; opencode/packages/opencode/src/session/processor.ts:330

Representative excerpt: codex/codex-rs/core/src/session/turn.rs:128

```
128: /// Takes initial turn input and runs a loop where, at each sampling request, 129: /// the model replies with either:
```

Builder Transfer

- Own prompt admission, active-turn serialization, continuation, cancellation, and finalization as explicit state transitions.

- Record tool requests or assistant tool-call turns before executing tools so crash recovery and replay have ground truth.
- It changes continuation semantics, avoids duplicated user turns, and must preserve routing and tool-result invariants.

Chapter 5: Tool Systems And Permission Boundaries

Thesis: A tool is a capability with schema, policy, lifecycle hooks, result settlement, telemetry, and user-facing metadata.

All six systems split model-visible tool descriptions from executable handlers. Codex routes specs through a ToolRouter and handler registry. OpenCode materializes tools per session/model/permission set. Goose keeps tools MCP-shaped and categorizes frontend/backend actions. OpenClaw assembles coding tools, plugin tools, channel tools, and Tool Search before applying layered policy. NanoClaw exposes container MCP tools while the host owns channels, ACLs, approvals, and credentials. Hermes uses a central registry and executor that can run calls sequentially or concurrently under approval and interrupt rules.

The permission boundary is not only the approval modal. It starts when the harness decides which tools are even visible. It continues through argument validation, stale-call checks, path normalization, cwd resolution, sandbox choice, output truncation, redaction, and persistence. The model should never be the component deciding whether a high-authority action is safe.

The right mental model is an API gateway. A model request arrives. The harness authenticates capability, validates input, applies static policy, requests dynamic approval when needed, executes through a controlled adapter, records telemetry, transforms output, and returns a bounded result. If each tool implements these concerns differently, safety and debuggability decay quickly.

Practices

- Keep a single registry path for lookup, validation, authorization, execution, and result settlement.
- Attach human-readable metadata to dangerous calls before asking for approval.
- Differentiate provider-executed tools from locally executed tools in both transcript and telemetry.

Code Walk-Through

- E03: Tools need an execution registry distinct from model-visible specifications. Sources: codex/codex-rs/core/src/tools/registry.rs:401; opencode/packages/opencode/src/session/tools.ts:39
- E04: Permission systems are layered, not binary. Sources: codex/codex-rs/protocol/src/approvals.rs:134; goose/crates/goose/src/permission/permission_inspector.rs:56
- E05: Filesystem and network authority must be represented as data. Sources: codex/codex-rs/protocol/src/permissions.rs:79; codex/codex-rs/sandboxing/src/manager.rs:34
- E12: Repeated tool-call and malformed-tool recovery should be first-class. Sources: hermes-agent/agent/conversation_loop.py:3972; opencode/packages/opencode/src/session/processor.ts:330

Representative excerpt: codex/codex-rs/core/src/tools/registry.rs:401

```
401: #[expect( 402: clippy::await_holding_invalid_type,
```

Builder Transfer

- Separate advertised schemas from executable handlers, permissions, lifecycle hooks, and result persistence.
- Combine static policy, dynamic approval, model/judge assistance, and post-policy security inspectors.

- Expose sandbox kind, writable roots, network state, external directory approvals, and path provenance as typed runtime facts.

Chapter 6: Context Engineering And Memory

Thesis: Context is maintained runtime state rendered into prompts at the last responsible moment.

The repositories treat context as more than conversation text. Codex tracks history, world state, image stripping, rollback, compaction, skills, plugins, and instruction fragments. OpenCode uses context epochs and baselines. OpenClaw has context engines, prompt-cache boundaries, memory contributions, bootstrap files, and channel-specific prompt surfaces. Goose computes compaction thresholds and has prompt manager snapshots. Hermes rebuilds or preserves system prompts according to compression and cache constraints. NanoClaw formats database messages into XML-like routing context with timezone, sender, destination, and system responses.

Memory follows the same rule. The problem is not how to append more text. The problem is routing: owner, scope, load phase, cacheability, size bound, invalidation, and evidence. A skill loaded for a spreadsheet task should not become permanent global context. A channel instruction should not silently override a system safety rule. A retrieved memory should be traceable enough that later debugging can answer why it appeared.

Compaction is the control-flow edge of context engineering. It changes what the next provider call sees and can duplicate or lose user turns if the runtime is careless. Strong systems model compaction as a transition with a reason, retained tail, summary artifact, and continuation semantics.

Practices

- Store context facts separately from their provider-specific prompt rendering.
- Make memory sources explicit by owner, scope, load phase, and maximum budget.
- Treat compaction summaries as runtime artifacts that need evidence preservation and duplicate-turn protection.

Code Walk-Through

- E06: Context is a maintained runtime object, not a concatenated prompt string. Sources: codex/codex-rs/core/src/context_manager/history.rs:36; opencode/packages/core/src/session/context-epoch.ts:46
- E07: Compaction is a control path, not a background cleanup task. Sources: codex/codex-rs/core/src/session/turn.rs:345; goose/crates/goose/src/context_mgmt/mod.rs:1
- E19: Prompt architecture is rendered from layers with cache and hierarchy constraints. Sources: codex/codex-rs/core/gpt_5_codex_prompt.md:1; goose/crates/goose/src/agents/prompt_manager.rs:11

Representative excerpt: codex/codex-rs/core/src/context_manager/history.rs:36

```
36: /// Transcript of thread history 37: #[derive(Debug, Clone, Default)]
```

Builder Transfer

- Track baselines, epochs, world state, prompt-cache boundaries, tool schemas, and compacted summaries explicitly.
- It changes continuation semantics, avoids duplicated user turns, and must preserve routing and tool-result invariants.
- System prompts, instruction files, skills, runtime facts, prompt-cache boundaries, and channel hints should be assembled by code, not copied into one giant string.

Chapter 7: Guardrails, Safety, And Human Approval

Thesis: Production guardrails are layered, explainable, and fail closed when provenance or authority is ambiguous.

The code rejects a single global allow/deny switch. Codex derives filesystem/network policy from permission profiles and routes risky actions through approvals or guardian assessment. Goose combines user permission levels, read-only classification, SmartApprove, and security inspectors. OpenCode defaults to ask and persists project-scoped approvals. OpenClaw composes profile, provider, global, agent, group, sender, sandbox, subagent, and inherited policies. NanoClaw keeps high-trust enforcement on the host outside the untrusted container. Hermes centralizes dangerous command approval with hardline never-run patterns, secret scopes, and redaction.

Human approval works only when the user sees the real action. A useful approval request names the command or tool, cwd, target path, host or sandbox, network need, credential class, and risk. A generic 'allow tool' prompt is not enough for a coding harness that can mutate files, run shell commands, send messages, or invoke cloud APIs.

Safety also has to survive product growth. Adding a desktop UI, channel plugin, MCP server, mobile client, or scheduler should not bypass the same policy engine. The policy result should be a stored fact that can explain what happened later.

Practices

- Separate policy resolution from approval UI, then make the UI a projection of the resolved risk.
- Record the source layer that allowed, denied, or escalated each tool request.
- Keep secrets, filesystem escape, network access, and destructive commands below model-controlled branches.

Code Walk-Through

- E04: Permission systems are layered, not binary. Sources: codex/codex-rs/protocol/src/approvals.rs:134; goose/crates/goose/src/permission/permission_inspector.rs:56
- E05: Filesystem and network authority must be represented as data. Sources: codex/codex-rs/protocol/src/permissions.rs:79; codex/codex-rs/sandboxing/src/manager.rs:34
- E09: Secret handling belongs below the agent interface. Sources: codex/codex-rs/core/src/config/auth_keyring.rs:10; nanoclaw/src/modules/approvals/onecli-approvals.ts:1

Representative excerpt: codex/codex-rs/protocol/src/approvals.rs:134

```
134: #[derive(Debug, Clone, Deserialize, Serialize, PartialEq, JsonSchema, TS)] 135: #[serde(tag = "type", rename_all = "snake_case")]
```

Builder Transfer

- Combine static policy, dynamic approval, model/judge assistance, and post-policy security inspectors.
- Expose sandbox kind, writable roots, network state, external directory approvals, and path provenance as typed runtime facts.
- Agents should see handles, scoped access, redaction, approval cards, or injected request credentials, not raw secrets.

Chapter 8: File Editing, Shell Execution, And Browser/Computer Control

Thesis: High-authority tools need specialized workflows because their failure modes are different.

File editing is not generic string replacement. Codex has a dedicated apply_patch parser and handler path. OpenCode has both exact edit and apply_patch tools with mutation and permission layers. OpenClaw uses edit tools, queued file mutation, diff rendering, patch parsing, and guarded host/sandbox filesystem operations. Goose has developer file read/write/edit primitives. Hermes isolates ACP edit approval helpers so UI-bound edit requests can be approved before mutation. These designs make diffs, paths, and write authority explicit.

Shell execution is even sharper. Codex unified exec chooses cwd, sandbox type, env, approval, timeouts, and output budgets. OpenCode documents host authority and gates it with permissions and workdir controls. Goose exposes structured shell output and truncation. OpenClaw adds exec host targeting, safe-bin policy, and gateway/native approvals. Hermes supports multiple execution backends while routing dangerous commands through centralized approval.

Browser and computer control need a third pattern. Goose includes a computer controller MCP. Hermes browser tools choose cloud or local browser backends and expose accessibility-tree style page state. OpenClaw starts sandbox browser containers, browser bridges, plugin SDK facades, and security audits for browser sandbox exposure. These tools have visual state, cookies, tabs, screenshots, CDP endpoints, and network exposure that cannot be reduced to a normal function call.

Practices

- Represent file mutation as patch/edit intent, preview, approval, write, diff, and persisted result.
- Define shell authority as a contract over cwd, env, timeout, network, sandbox, output budget, and approval.
- Treat browser/computer control as stateful infrastructure with lifecycle cleanup and audit tests.

Code Walk-Through

- E10: Shell execution is a product decision, not a generic subprocess wrapper. Sources: codex/codex-rs/core/src/tools/handlers/unified_exec/exec_command.rs:80; goose/crates/goose/src/agents/platform_extensions/developer/shell.rs:351
- E16: File editing is a structured mutation workflow, not an arbitrary text rewrite. Sources: codex/codex-rs/core/src/tools/handlers/apply_patch.rs:1; goose/crates/goose/src/agents/platform_extensions/developer/edit.rs:1
- E17: Browser and computer-use tools need their own lifecycle and audit boundary. Sources: goose/crates/goose-mcp/src/computercontroller/mod.rs:1; hermes-agent/tools/browser_tool.py:1

Representative excerpt: codex/codex-rs/core/src/tools/handlers/unified_exec/exec_command.rs:80

```
80: impl ToolExecutor<ToolInvocation> for ExecCommandHandler { 81: fn tool_name(&self) -> ToolName {
```

Builder Transfer

- Execution paths encode cwd rules, timeouts, output budgets, sandbox preference, approval policy, and environment filtering.

- Exact edits and patches should pass through parsers, mutation queues, approval hooks, diffs, and guarded filesystem writes.
- The harness should isolate browser state, expose screenshots or accessibility trees intentionally, and audit sandbox/container exposure.

Chapter 9: Multi-Agent Orchestration Patterns

Thesis: Subagents are separate units of work, not just longer prompts with a role label.

The repos show several delegation shapes. Codex has delegate/code-mode paths and rollout trace support for subagent events. Goose runs subagent tasks through a handler that builds child prompts, extension context, task config, and notification events. OpenCode defines subagent permission rules so child sessions do not inherit everything blindly. OpenClaw has one of the richest implementations: spawn validation, child sessions, attachments, depth limits, ownership, target policy, registry state, liveness, delivery, and reactivation. Hermes dispatches `delegate_task` through a single call site, caps concurrent children, tracks depth, and returns handles/results into the parent flow. NanoClaw is different: its cited path creates long-lived companion agents in a unified destination namespace, with host-side authorization, rather than proving a full child-run orchestration stack.

The common mistake is to let the parent model launch arbitrary children without lifecycle ownership. A production harness needs to know who requested the subagent, what workspace it owns, what tools it may use, how results re-enter the parent, when it times out, and whether the user can inspect the child transcript.

Subagents are most useful when they reduce contention in the main context. They can explore code, run bounded verification, produce adversarial review, or own a narrow patch area. They are harmful when they duplicate the parent task, hide authority, or return summaries without evidence.

Practices

- Give each child run an owner, target, depth, budget, permission set, and result channel.
- Make subagent completion, failure, and notification events visible to the parent and UI.
- Prevent recursive delegation from escaping the same policy and cost controls as the parent run.

Code Walk-Through

- E18: Subagent orchestration is strongest when child work has ownership, limits, permissions, and result re-entry. Sources: openclaw/src/agents/subagent-spawn.ts:161; hermes-agent/run_agent.py:5231

Representative excerpt: openclaw/src/agents/subagent-spawn.ts:161

```
161: export type SpawnSubagentParams = { 162: task: string;
```

Builder Transfer

- Full child-run orchestration needs depth limits, child-session context, permission inheritance rules, delivery state, and explicit summarization or handoff points; NanoClaw shows a narrower long-lived agent creation/destination pattern.

Chapter 10: Prompt Architecture And Instruction Hierarchy

Thesis: Production prompts are rendered from ordered, inspectable layers with cache, scope, and conflict rules.

Prompt architecture is code. Codex has model-specific prompt files, generated environment context, skill/plugin injection, and AGENTS.md loading. Goose assembles prompts through a prompt manager and ships system, plan, compaction, permission-judge, recipe, and subagent prompt assets. OpenCode builds session prompts from system and instruction services. OpenClaw assembles runtime, workspace, tooling, memory, delegation, channel, and cache-boundary sections. NanoClaw renders structured message batches with routing context and system responses. Hermes builds the system prompt once per session, preserves cache invariants, and adds skills, context files, platform hints, and toolset guidance.

The hierarchy matters because instructions conflict. User text, repo instructions, skill instructions, tool output, channel formatting, and developer policy do not have the same authority. The harness must encode that hierarchy outside the model's discretion. It should also make prompt-cache boundaries intentional: dynamic content should not accidentally invalidate expensive stable prefixes.

A practical prompt architecture has two artifacts: the structured sources and the final rendered provider request. Debugging needs both. The final request explains what the model saw; the sources explain why it was there and how to fix the renderer.

Practices

- Separate instruction source loading from provider request rendering.
- Annotate prompt layers by authority, scope, cacheability, and invalidation trigger.
- Test prompt rendering around cache boundaries and dynamic tool/context changes.

Code Walk-Through

- E06: Context is a maintained runtime object, not a concatenated prompt string. Sources: codex/codex-rs/core/src/context_manager/history.rs:36; opencode/packages/core/src/session/context-epoch.ts:46
- E19: Prompt architecture is rendered from layers with cache and hierarchy constraints. Sources: codex/codex-rs/core/gpt_5_codex_prompt.md:1; goose/crates/goose/src/agents/prompt_manager.rs:11

Representative excerpt: codex/codex-rs/core/src/context_manager/history.rs:36

```
36: /// Transcript of thread history 37: #[derive(Debug, Clone, Default)]
```

Builder Transfer

- Track baselines, epochs, world state, prompt-cache boundaries, tool schemas, and compacted summaries explicitly.
- System prompts, instruction files, skills, runtime facts, prompt-cache boundaries, and channel hints should be assembled by code, not copied into one giant string.

Chapter 11: Testing, Evals, Observability, And Debugging

Thesis: Confidence comes from testing runtime boundaries, evaluating live behavior, and tracing sanitized production events.

The test surfaces mirror the harness boundaries. Codex has Rust tests, prompt/debug tests, TUI snapshots, MCP/tool exposure tests, and OpenTelemetry tests. Goose has Rust tests, UI tests, self-test recipes, Harbor eval runner, open-model-gym scenarios, and LLM judge postprocessing scripts. OpenCode has Bun tests for session runner, tools, permissions, patching, app behavior, and observability. OpenClaw has extensive unit, gateway, plugin, channel, sandbox, browser, install, performance, stability, and telemetry tests. NanoClaw covers DB, channel, skills, and container behaviors. Hermes slices Python and JS tests while documenting observer hooks and redaction expectations.

Evals and observability answer different questions. Evals ask whether a model/tool combination produces acceptable task outcomes. Observability asks what happened in a specific run: provider usage, tool calls, approvals, failures, fallback, delivery, cache behavior, and redacted payloads. Debugging needs both plus deterministic regression tests for policy, parsers, compaction, and replay.

The most important production habit is to test the promises the harness makes. If the harness promises not to leak secrets, test redaction. If it promises one active run per session, test duplicate drains. If it promises resumability, kill the process between persisted intent and side effect settlement. If it promises browser isolation, audit container exposure.

Practices

- Build the test matrix around invariants: permissions, path resolution, parser correctness, settlement, replay, compaction, delivery, redaction, and fallback.
- Run live provider evals separately from deterministic CI and label cost/secrets requirements clearly.
- Emit sanitized telemetry at run, provider, tool, approval, and delivery layers.

Code Walk-Through

- E14: Test strategy follows harness boundaries. Sources: codex/codex-rs/core/tests/suite/otel.rs:113; goose/goose-self-test.yaml:66
- E20: Evaluation and observability are separate but connected feedback loops. Sources: goose/scripts/bench-postprocess-scripts/llm-judges/llm_judge.py:74; openclaw/src/agents/embedded-agent-runner/prompt-cache-observability.ts:158

Representative excerpt: codex/codex-rs/core/tests/suite/otel.rs:113

```
113: #[tokio::test] 114: #[traced_test]
```

Builder Transfer

- Unit tests cover policy and parsers; integration and e2e tests cover session loops, UI surfaces, provider behavior, and install/runtime smoke.
- Deterministic tests, live evals, judge scripts, telemetry spans, prompt-cache diagnostics, and sanitized observer hooks each answer different production questions.

Chapter 12: Product UX For Agentic Systems

Thesis: The UI should expose the harness state machine rather than hiding everything behind a single thinking indicator.

The mature systems make runtime state visible. Codex TUI handles history, exec, approvals, onboarding, resume, and status. Goose desktop routes cover chat, settings, extensions, apps, sessions, schedules, recipes, skills, permissions, and launcher flows. OpenCode's app consumes SDK events, reconnects, heartbeats, permissions, prompt state, and optimistic updates. OpenClaw tracks partial replies, reasoning, compaction, tool metadata, deterministic approval prompts, media, and deferred delivery. NanoClaw's host owns delivery drains, retries, edits, reactions, files, and questions. Hermes has web, desktop, TUI, gateway, and messaging surfaces.

Product UX is not separate from runtime design. If a UI cannot distinguish waiting for approval from provider stall, users will interrupt the wrong thing. If it cannot replay state after reconnect, a long-running task becomes untrustworthy. If delivery records are not typed, channels can double-send or lose files.

A good agent UI is a projection of stored facts: state, active tool, approval request, command output, token/cost usage, file diffs, delivery status, child-agent progress, and recoverable errors. When those facts are missing from the runtime, the UI compensates with guesswork.

Practices

- Expose runtime states as typed events before building UI widgets around them.
- Make approval cards exact: command/tool, cwd, path, host, network/credential need, and risk.
- Design reconnect and replay as API guarantees, not frontend cache behavior.

Code Walk-Through

- E08: Product UX is part of the harness. Sources: codex/codex-rs/tui/src/main.rs:50; goose/ui/desktop/src/App.tsx:598
- E11: Agent systems need message delivery semantics, not just chat transcripts. Sources: nanoclaw/src/router.ts:1; nanoclaw/src/delivery.ts:1

Representative excerpt: codex/codex-rs/tui/src/main.rs:50

```
50: fn main() -> anyhow::Result<()> { 51: arg0_dispatch_or_else(|arg0_paths: Arg0DispatchPaths| async move {
```

Builder Transfer

- Stream reasoning, tool metadata, approvals, partial replies, delivery evidence, snapshots, and resumable state to users and clients.
- Multi-surface agents need routing, destinations, delivery receipts, edits, reactions, and explicit final-addressing rules.

Chapter 13: Design Patterns Worth Copying

Thesis: The strongest reusable patterns are durable intent, narrow registries, layered policy, bounded context, and evented product state.

The first pattern is durable-before-dangerous. Store input, tool intent, call IDs, and continuation records before executing side effects. This shows up in OpenCode, Hermes, NanoClaw, and Codex and is the backbone of crash recovery, replay, audit, and support.

The second pattern is one narrow route for each high-risk boundary. One tool registry. One permission resolver. One shell contract. One file mutation path. One browser lifecycle manager. One secret gateway. Narrow routes do not mean little code; they mean all code must pass through a place where policy, telemetry, and result shaping are consistent.

The third pattern is product state as runtime state. Approvals, tool progress, compaction, delivery, provider fallback, and subagent completion should be first-class events. This is how terminal, desktop, web, channel, and API surfaces stay consistent.

Practices

- Persist intent before side effects.
- Centralize policy and execution wrappers at each authority boundary.
- Design surfaces as clients of stored runtime events.

Code Walk-Through

- E02: Durability starts before side effects. Sources: opencode/packages/core/src/session/runner/llm.ts:66; hermes-agent/agent/conversation_loop.py:4203
- E03: Tools need an execution registry distinct from model-visible specifications. Sources: codex/codex-rs/core/src/tools/registry.rs:401; opencode/packages/opencode/src/session/tools.ts:39
- E04: Permission systems are layered, not binary. Sources: codex/codex-rs/protocol/src/approvals.rs:134; goose/crates/goose/src/permission/permission_inspector.rs:56
- E08: Product UX is part of the harness. Sources: codex/codex-rs/tui/src/main.rs:50; goose/ui/desktop/src/App.tsx:598
- E15: A smaller core with a wider edge is the dominant shape. Sources: codex/codex-rs/core/src/session/session.rs:51; goose/crates/goose/src/agents/agent.rs:236

Representative excerpt: opencode/packages/core/src/session/runner/llm.ts:66

```
66: * - Tool settlement and continuation 67: * - [x] Durably record each tool call before side effects begin.
```

Builder Transfer

- Record tool requests or assistant tool-call turns before executing tools so crash recovery and replay have ground truth.
- Separate advertised schemas from executable handlers, permissions, lifecycle hooks, and result persistence.
- Combine static policy, dynamic approval, model/judge assistance, and post-policy security inspectors.

Chapter 14: Anti-Patterns And Failure Modes

Thesis: Most harness failures come from hidden authority, implicit state, unbounded context, and vague recovery.

The codebases contain the failure modes because they have grown past them. Invalid tool names, malformed JSON, repeated tool calls, truncated arguments, context overflow, stale provider continuations, stuck containers, duplicate delivery, path escapes, leaked text tool syntax, prompt-cache churn, and auth profile failures are all real runtime concerns. A harness that treats these as generic exceptions will give users poor recovery and operators poor evidence.

Several anti-patterns are easy to recognize. A tool function that writes files directly without diff/approval/settlement is too much authority in one place. A prompt that contains permanent memory, current channel rules, and secret hints in one string is too hard to audit. A UI that polls logs instead of consuming events is inventing state. An eval script that needs production secrets in normal CI is not a safe regression lane.

The hardest anti-pattern is confidence without observability. Agents can appear to work until a crash, retry, compaction, or channel send exposes missing state. The systems studied here invest heavily in explicit states because that is where production failure actually lands.

Practices

- Classify failures with vocabulary the runtime and UI both understand.
- Do not let provider retries duplicate local side effects.
- Avoid unbounded tool output and memory injection into the next model request.

Code Walk-Through

- E07: Compaction is a control path, not a background cleanup task. Sources: codex/codex-rs/core/src/session/turn.rs:345; goose/crates/goose/src/context_mgmt/mod.rs:1
- E12: Repeated tool-call and malformed-tool recovery should be first-class. Sources: hermes-agent/agent/conversation_loop.py:3972; opencode/packages/opencode/src/session/processor.ts:330
- E13: A mature harness is observable at the run, tool, provider, and delivery layers. Sources: codex/codex-rs/otel/src/provider.rs:77; openclaw/src/infra/diagnostic-events.ts:1
- E20: Evaluation and observability are separate but connected feedback loops. Sources: goose/scripts/bench-postprocess-scripts/llm-judges/llm_judge.py:74; openclaw/src/agents/embedded-agent-runner/prompt-cache-observability.ts:158

Representative excerpt: codex/codex-rs/core/src/session/turn.rs:345

```
345: // as long as compaction works well in getting us way below the token limit, we shouldn't worry about being in an inf...
346: if needs_follow_up
```

Builder Transfer

- It changes continuation semantics, avoids duplicated user turns, and must preserve routing and tool-result invariants.
- Detect invalid names, truncated JSON, doom loops, leaked text tool syntax, and unsettled provider calls as specific harness states.

- Record usage, lifecycle events, diagnostic states, failover reasons, stuck detection, and sanitized trajectories.

Chapter 15: Blueprint For Building A Production-Grade Coding Agent

Thesis: Build the smallest reliable coordinator first, then grow tools, policy, context, surfaces, evals, and operations around it.

Start with a durable schema: sessions, runs, turns, steps, tool calls, approvals, deliveries, outputs, provider usage, and errors. Then write the active-run coordinator that admits input, builds context, materializes tools through policy, streams one provider turn, persists tool calls before side effects, executes through wrappers, settles results, and continues only for explicit reasons.

Next add authority boundaries in this order: filesystem/path policy, shell execution contract, file mutation path, network and browser lifecycle, secrets gateway, and external connectors. Do not add broad tool catalogs until these boundaries exist. Every new tool should inherit registry lookup, approval, telemetry, truncation, redaction, and persistence.

Finally, make the product and operating system catch up to the runtime: typed events for UI and API surfaces, replay/reconnect contracts, deterministic tests for invariants, live evals for model behavior, telemetry for run/tool/provider/delivery layers, and documented deployment/debug scripts. The model can improve later; the harness has to be trustworthy from the first destructive tool.

Practices

- Implement one durable loop with one provider and a minimal tool registry before adding surfaces.
- Gate high-authority actions with typed policy and human-readable approval requests.
- Ship with tests and telemetry for the boundaries most likely to fail in production.

Code Walk-Through

- E01: A production agent is a session coordinator, not just a while-loop around tool calls. Sources: codex/codex-rs/core/src/session/turn.rs:128; goose/crates/goose/src/agents/agent.rs:1888
- E03: Tools need an execution registry distinct from model-visible specifications. Sources: codex/codex-rs/core/src/tools/registry.rs:401; opencode/packages/opencode/src/session/tools.ts:39
- E04: Permission systems are layered, not binary. Sources: codex/codex-rs/protocol/src/approvals.rs:134; goose/crates/goose/src/permission/permission_inspector.rs:56
- E05: Filesystem and network authority must be represented as data. Sources: codex/codex-rs/protocol/src/permissions.rs:79; codex/codex-rs/sandboxing/src/manager.rs:34
- E06: Context is a maintained runtime object, not a concatenated prompt string. Sources: codex/codex-rs/core/src/context_manager/history.rs:36; opencode/packages/core/src/session/context-epoch.ts:46
- E14: Test strategy follows harness boundaries. Sources: codex/codex-rs/core/tests/suite/otel.rs:113; goose/goose-self-test.yaml:66
- E20: Evaluation and observability are separate but connected feedback loops. Sources: goose/scripts/bench-postprocess-scripts/llm-judges/llm_judge.py:74; openclaw/src/agents/embedded-agent-runner/prompt-cache-observability.ts:158

Representative excerpt: codex/codex-rs/core/src/session/turn.rs:128

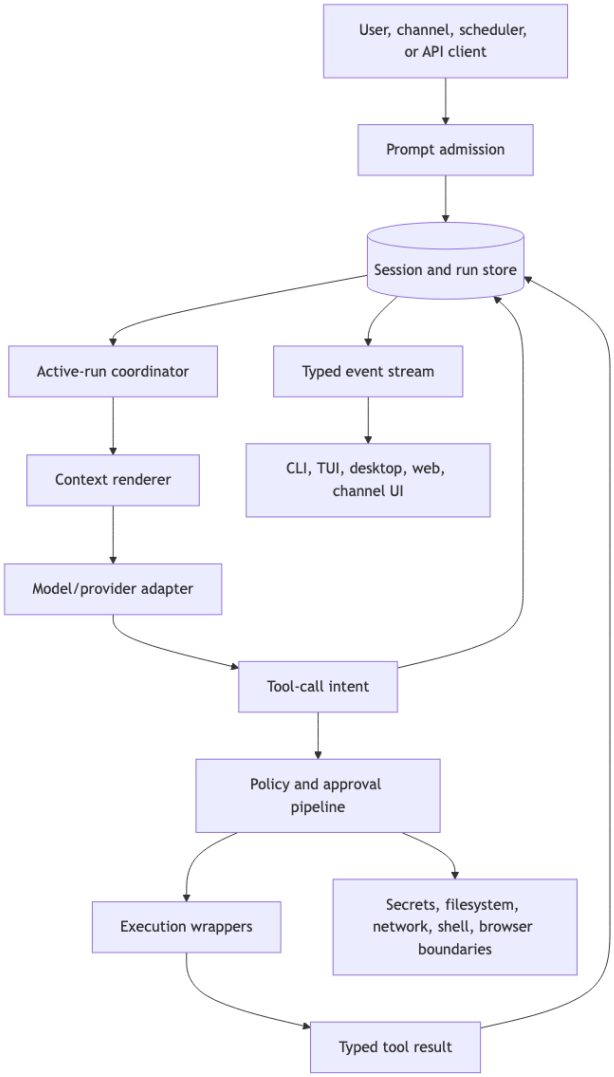
```
128: /// Takes initial turn input and runs a loop where, at each sampling request, 129: /// the model replies with either:
```

Builder Transfer

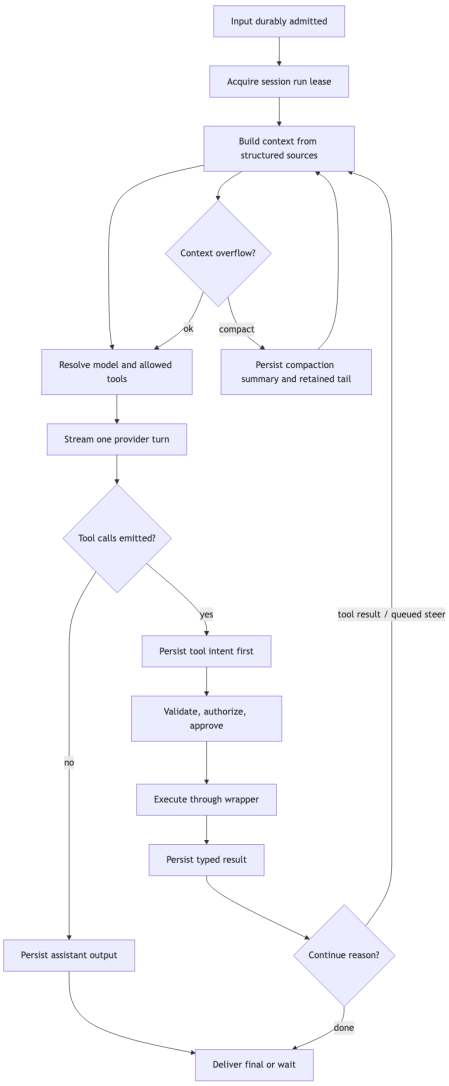
- Own prompt admission, active-turn serialization, continuation, cancellation, and finalization as explicit state transitions.
- Separate advertised schemas from executable handlers, permissions, lifecycle hooks, and result persistence.
- Combine static policy, dynamic approval, model/judge assistance, and post-policy security inspectors.

Diagrams

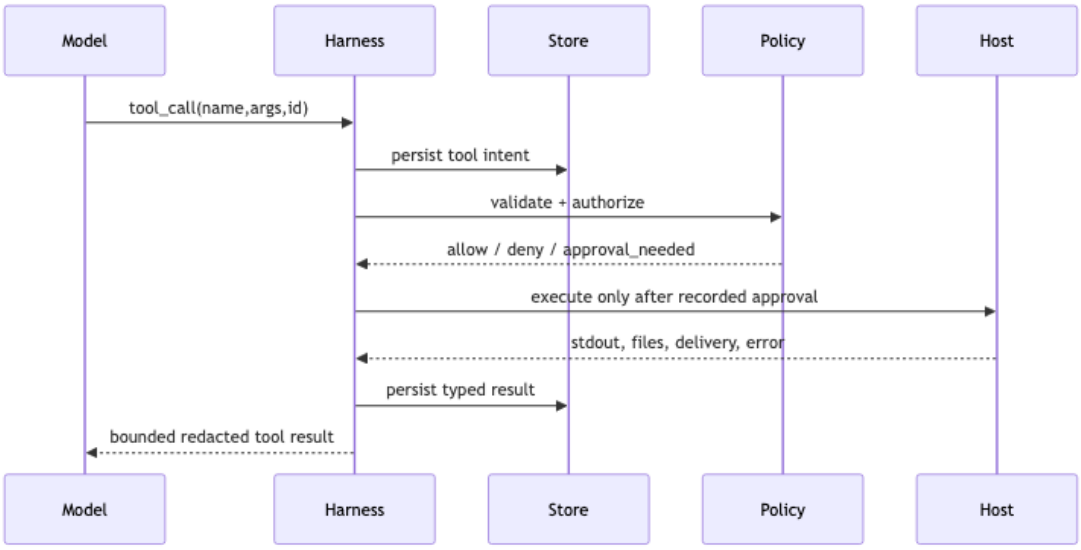
Production Agent Architecture



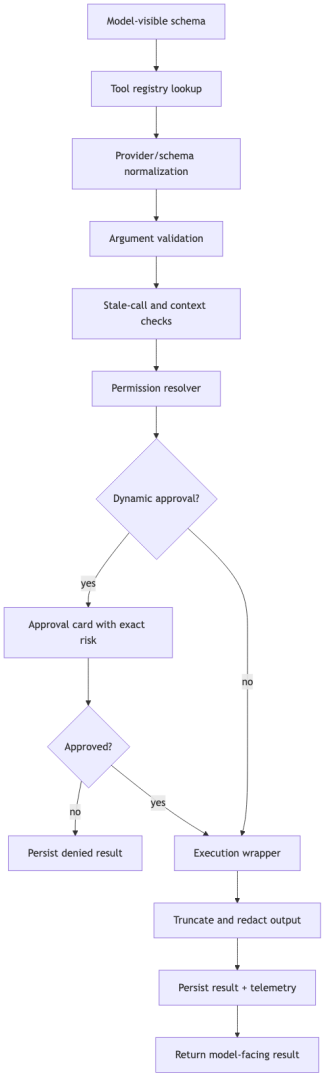
Agent Loop Lifecycle



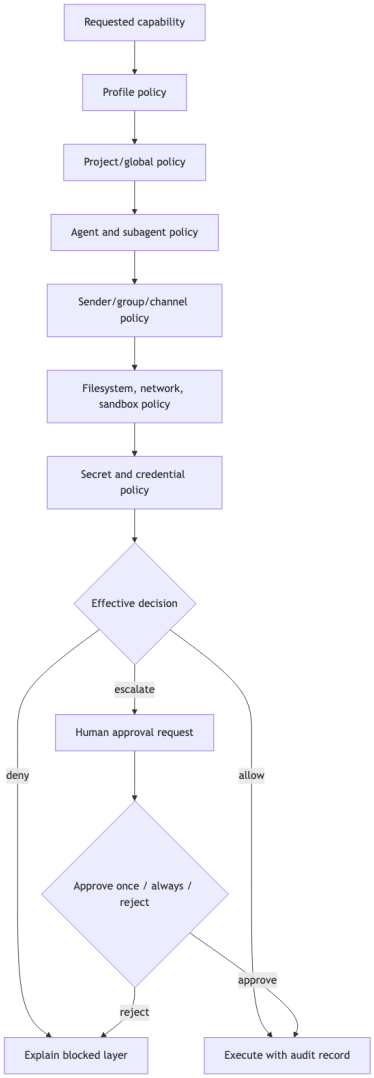
Persistence Before Side Effects



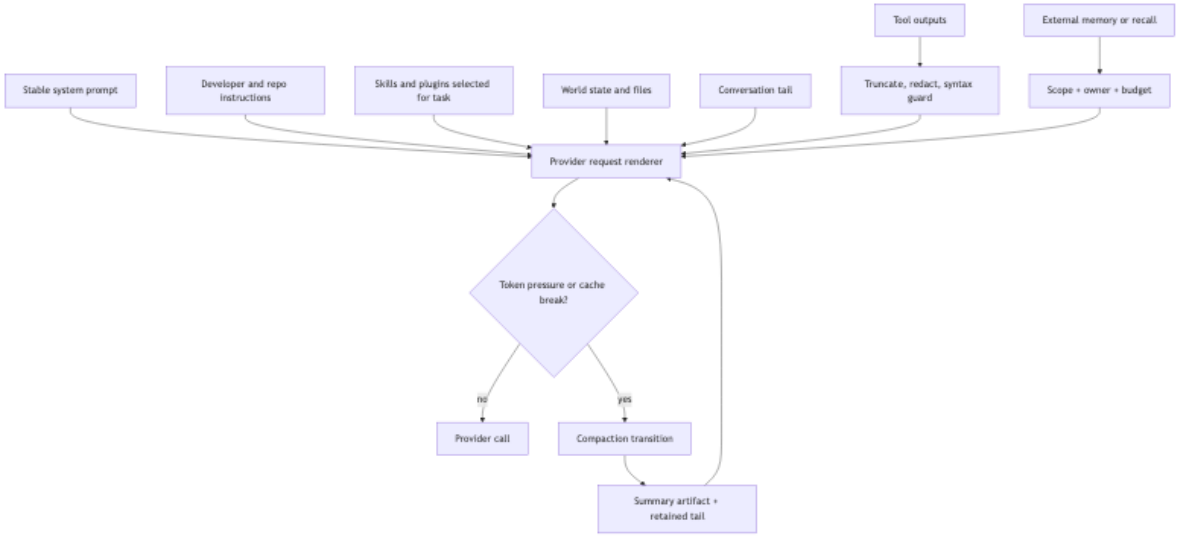
Tool Invocation Flow



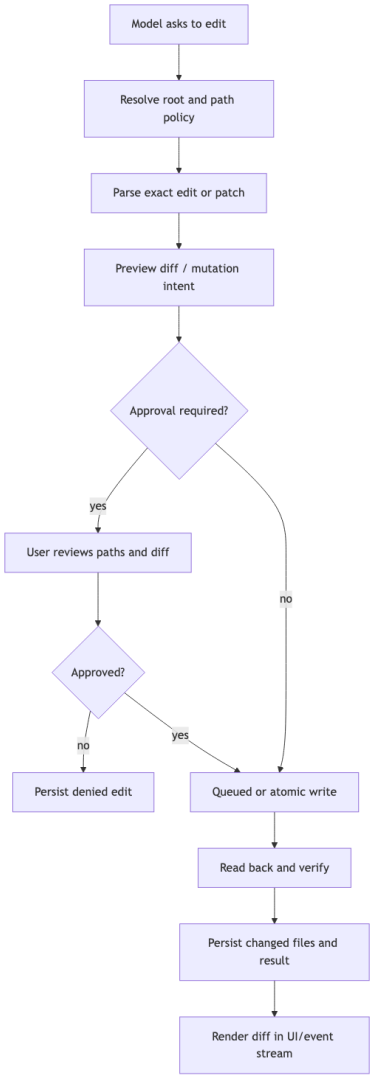
Permission Approval Flow



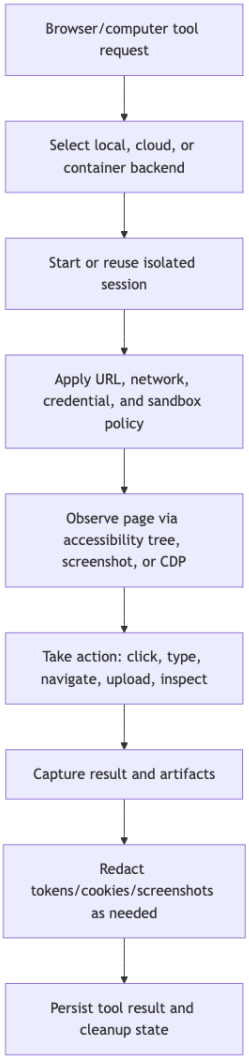
Memory Context Pipeline



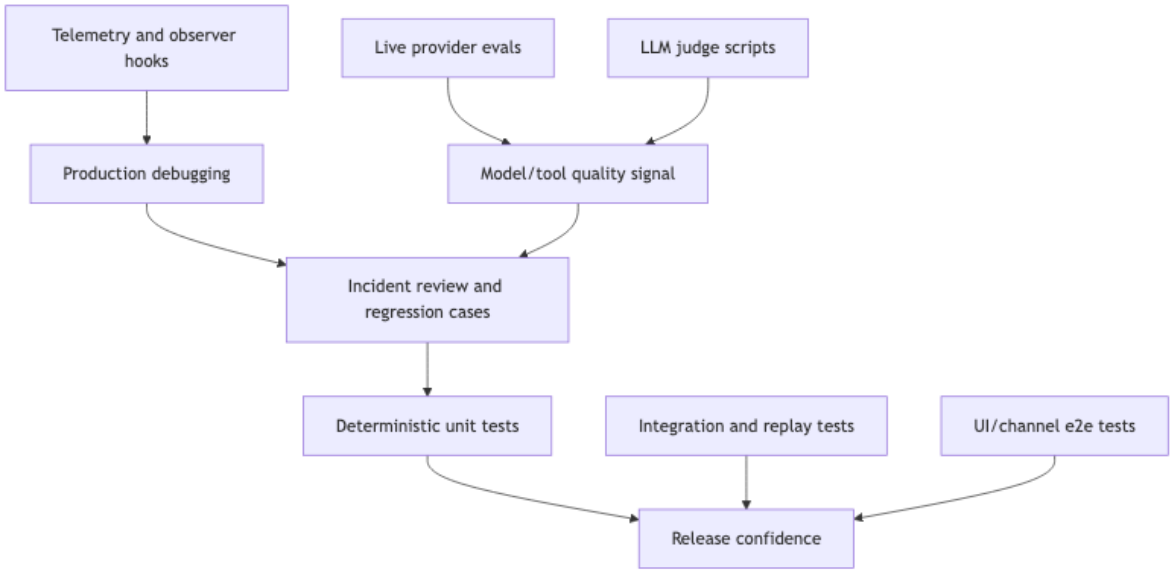
File Editing Workflow



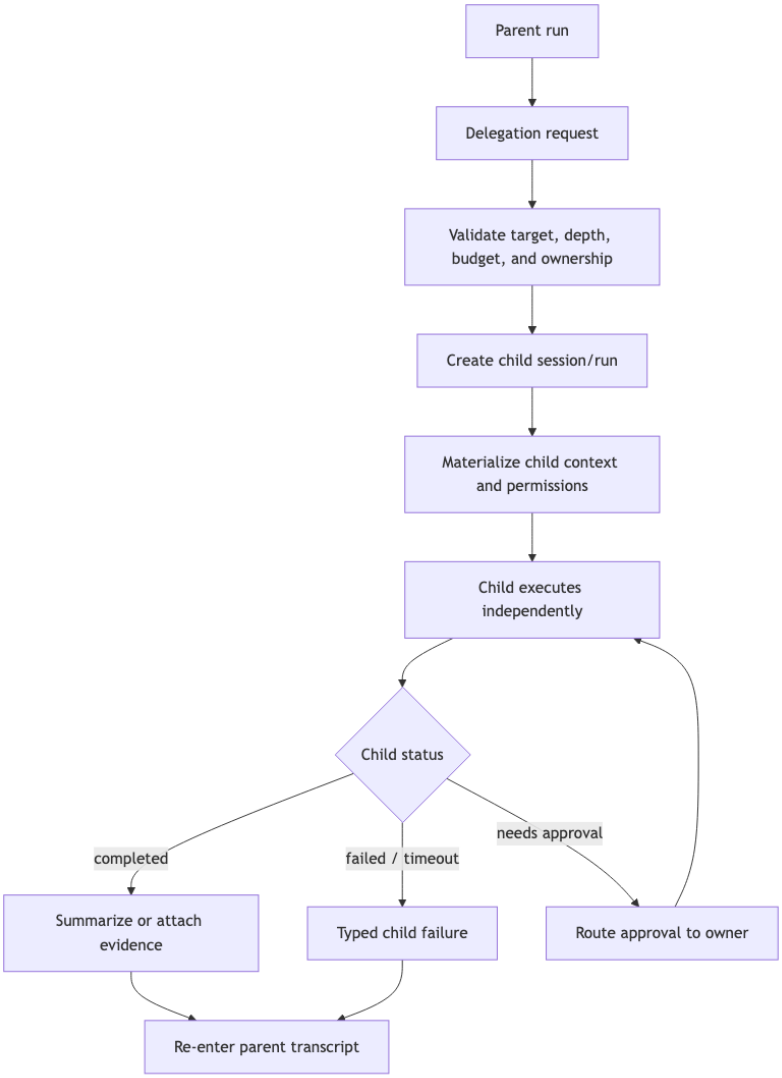
Browser Computer Use Workflow



Evaluation Observability Pipeline



Multi Agent Orchestration



Capability Matrix

Capability	Codex	Goose	OpenCode	OpenClaw	NanoClaw	Hermes	Evidence
Durable run loop	Strong	Strong	Strong	Strong	Strong	Strong	E01,E02
Tool registry lifecycle	Strong	Strong	Strong	Strong	Partial	Strong	E03,E12
Layered permissions	Strong	Strong	Strong	Strong	Strong	Strong	E04,E05
Sandboxing / host isolation	Strong	Partial	Partial	Strong	Strong	Partial	E05,E10
Secret boundary	Strong	Partial	Partial	Strong	Strong	Strong	E09
Context and compaction	Strong	Strong	Strong	Strong	Partial	Strong	E06,E07
File mutation workflow	Strong	Strong	Strong	Strong	Limited	Strong	E16
Browser / computer control	Limited	Strong	Limited	Strong	Limited	Strong	E17
Delivery / channel semantics	Partial	Partial	Partial	Strong	Strong	Strong	E11
Subagents	Strong	Strong	Strong	Strong	Partial	Strong	E18
Prompt hierarchy	Strong	Strong	Strong	Strong	Strong	Strong	E19
Testing / evals / ops	Strong	Strong	Strong	Strong	Partial	Strong	E14,E20

Builder Blueprint Artifacts

Durable schema for a minimal production coding agent.

Table	Purpose	Minimum fields
sessions	Long-lived conversation or task container	id, user_id, workspace_id, status, active_run_id, created_at, updated_at
runs	One execution drain for a session	id, session_id, state, lease_owner, started_at, completed_at, cancel_reason
turns	Durable user/assistant/system transcript units	id, session_id, run_id, role, content_ref, context_epoch, created_at
context_epochs	Versioned context baseline and compaction boundary	id, session_id, source_digest, summary_ref, retained_tail_ref, token_estimate
tool_calls	Model-emitted capability intent	id, run_id, turn_id, tool_name, args_json, policy_state, approval_id, status
tool_results	Settled tool outcome	id, tool_call_id, status, result_ref, error_kind, redaction_state, output_tokens
approvals	Human or policy escalation record	id, tool_call_id, risk, prompt_ref, decision, approver_id, expires_at
deliveries	Outbound product/channel messages and files	id, session_id, run_id, target, kind, payload_ref, status, external_id
provider_events	Provider stream, usage, and fallback facts	id, run_id, provider, model, event_type, usage_json, error_kind
audit_events	Append-only safety and operations trail	id, actor, action, subject_type, subject_id, policy_snapshot_ref, created_at

Coordinator Pseudocode

```
admit input -> acquire run lease -> render context -> materialize allowed tools -> stream one provider turn -> persist tool intent -> approve/execute -> settle result -> continue or finalize
```

Event Contract

Event	Meaning	Core fields
run.started	Run lease acquired and context build begins	session_id, run_id, lease_owner
context.rendered	Provider request context assembled	context_epoch, token_estimate, source_digests
provider.delta	Model stream produced text or tool intent	provider, model, delta_kind, sequence
tool.intent.persisted	Tool call recorded before side effect	tool_call_id, tool_name, args_digest
approval.requested	Human decision required	approval_id, risk, display_command, display_paths
tool.result	Tool settled	tool_call_id, status, error_kind, output_ref
delivery.updated	Outbound message/file changed status	delivery_id, target, status, external_id
run.blocked	Coordinator cannot continue without external action	reason, recoverability, suggested_action
run.completed	Final state reached	status, usage, deliveries_count

Operator Runbook

Situation	Operator action	Guardrail
Deploy smoke	Run one read-only prompt, one denied command, one approved harmless command, and one replay/reconnect check before opening traffic.	Release is not live until runtime events and redaction checks pass.
Rollback	Keep the previous runtime artifact and schema compatibility window; stop new run admission, let active runs drain or mark them resumable, then switch the serving symlink/container.	Never roll back only the UI when event schemas changed.
Stuck run	Find runs with no provider/tool/delivery event beyond the SLA, inspect lease owner, cancel or requeue with an audit event, and preserve the transcript.	Alert when run.blocked or active lease age exceeds threshold.
Tool settlement gap	Query tool_calls without terminal tool_results, mark interrupted if host process died, and replay only idempotent tools.	Persist intent before side effects so this query is possible.

Situation	Operator action	Guardrail
Delivery incident	Check delivery records by target/status/external_id, dedupe inflight drains, retry failed transient sends, and never resend if an external receipt exists.	Delivery retries need idempotency keys or platform receipts.
Cost spike	Break down provider_events by model, prompt tokens, completion tokens, tool loops, and subagent fan-out; cap or pause expensive agents.	Tie alerts to run/session/workspace owners.
Secret exposure	Rotate the credential, invalidate logs/snapshots containing the token, add a redaction test fixture, and move future access behind a handle or gateway.	Do not paste secret values into incident notes.
Context overflow	Trigger compaction with retained tail, verify the latest user turn is not duplicated, and switch to a larger context model only with an explicit audit event.	Track context.rendered token estimates and compaction reasons.

Appendix A: Repo-By-Repo Architecture Notes

Repo	Stack	Surfaces	Loop	Guardrail
Codex	Rust 2024 workspace with Tokio/tracing, ratatui TUI, axum app-server, plus pnpm maintenance tooling and SDK packages.	CLI, interactive TUI, non-interactive exec, JSON-RPC app-server, MCP mode, remote exec server, desktop launcher, cloud tasks, SDKs.	Thread and turn processors feed `run_turn`, which compacts, captures world state, injects skills/plugins, builds tools, samples, dispatches calls, records outputs, and follows up until completion.	Permission profiles produce filesystem and network sandbox policy; approvals cover sandbox escape, network access, MCP calls, apply_patch, and command execution.
Goose	Rust workspace with Tokio, RMCP, Axum server APIs, and TypeScript Electron/React/Vite/Ink UI packages.	CLI, `goosed` server, Electron desktop, Ink text UI, MCP servers, app recipes, schedules, and Harbor eval runner.	The agent prepares conversation/tools/system prompt, streams provider output, categorizes frontend/backend tools, inspects permissions, dispatches streams, persists messages, and loops through tool results.	Modes include auto, approve, smart_approve, and chat; inspector stack includes security, egress, adversary, permission, and repetition.
OpenCode	TypeScript ESM monorepo on Bun workspaces, Effect services/layers, AI SDK adapters, Solid/Vite app, Electron desktop.	CLI run/serve/web/tui/acp/mcp, HTTP API, SSE events, PTY websocket routes, ACP server, VS Code terminal launcher, desktop sidecar.	V2 session core separates durable prompt admission from execution, then promotes steer/queue input, prepares system context, resolves model/tools, streams exactly one provider turn, settles local tools, and continues when needed.	Permissions default to ask, saved approvals are project scoped, path mutation blocks relative escapes, and shell authority is gated by permissions rather than a hard process sandbox.
OpenClaw	TypeScript/Node ESM monorepo using pnpm, Vite/Lit control UI, plugin runtime, Android Kotlin/Compose, and Swift shared libraries.	CLI, gateway HTTP/WS server, TUI, Control UI, channel plugins, plugin SDK, Android, iOS/macOS SwiftKit.	Embedded runs pass through command policy, auth profile selection, queue lanes, sandbox context, tool construction, context engine assembly, provider attempts, fallback, delivery, and cleanup.	Layered tool policy spans profile, provider, global, agent, group, sender, sandbox, subagent, and inherited policies; gateway exposure and exec approvals are explicitly guarded.
NanoClaw	TypeScript ESM host on Node with pnpm, Bun-based container runner, better-sqlite3, chat SDK bridge, OneCLI SDK.	Host daemon, `ncl` admin CLI, local CLI chat, Chat SDK bridge adapters, per-session agent containers, setup/service tooling.	Channel event routes to central DB and session inbound DB, wakes one session container, runner polls inbound rows, queries provider, writes outbound rows, and host delivery drains outbound DB.	Container is untrusted; host enforces sender access, command privileges, CLI scope, destination ACLs, approvals, mount security, egress lockdown, and OneCLI credential mediation.
Hermes	Python package for 3.11-3.13 with setuptools/uv-style locked installs, plus npm workspaces for React/Vite web, Ink TUI, Electron desktop.	CLI, direct runner, ACP stdio adapter, messaging gateway, MCP stdio server, TUI JSON-RPC gateway, dashboard, desktop.	AIAgent facade builds TurnContext, persists user input early, compresses, calls model with retry/fallback, validates tool calls, persists assistant tool-call turns before side effects, and dispatches tools sequentially or concurrently.	Dangerous command approval is centralized with hardline never-run patterns, smart approval, CLI/gateway approval paths, redaction, sensitive path refusal, and fail-closed secret scopes.

Appendix B: Evidence Matrix

ID	Claim	Observed behavior	Representative sources
E01	A production agent is a session coordinator, not just a while-loop around tool calls.	The cited session and loop files own turn admission, context construction, provider streaming, tool settlement, continuation, or finalization rather than delegating those to a prompt-only convention.	codex/codex-rs/core/src/session/turn.rs:128; goose/crates/goose/src/agents/agent.rs:1888; opencode/packages/core/src/session/runner/llm.ts:40
E02	Durability starts before side effects.	The cited runtimes record assistant/tool-call intent, continuation identifiers, or injected conversation facts before letting local tools or delivery paths change external state.	opencode/packages/core/src/session/runner/llm.ts:66; hermes-agent/agent/conversation_loop.py:4203; nanoclaw/container/agent-runner/src/poll-loop.ts:462
E03	Tools need an execution registry distinct from model-visible specifications.	The cited registries and MCP client paths separate model-facing schemas from executable handlers, lifecycle hooks, metadata, and per-session tool materialization.	codex/codex-rs/core/src/tools/registry.rs:401; opencode/packages/opencode/src/session/tools.ts:39; openclaw/src/agents/agent-tools.ts:546
E04	Permission systems are layered, not binary.	The cited approval and permission files compose static policy, read-only/tool classification, dynamic approval, sender or sandbox provenance, and security inspection into effective execution decisions.	codex/codex-rs/protocol/src/approvals.rs:134; goose/crates/goose/src/permission/permission_inspector.rs:56; opencode/packages/core/src/permission.ts:75
E05	Filesystem and network authority must be represented as data.	The cited permission and sandbox files model filesystem roots, network authority, location mutation, sandbox contexts, container mounts, and environment filtering as typed runtime facts.	codex/codex-rs/protocol/src/permissions.rs:79; codex/codex-rs/sandboxing/src/manager.rs:34; opencode/packages/core/src/location-mutation.ts:11
E06	Context is a maintained runtime object, not a concatenated prompt string.	The cited context files maintain history, epochs, prompt managers, context engines, formatter output, skills, world state, and compression boundaries as owned runtime data.	codex/codex-rs/core/src/context_manager/history.rs:36; opencode/packages/core/src/session/context-epoch.ts:46; openclaw/src/context-engine/types.ts:7
E07	Compaction is a control path, not a background cleanup task.	The cited compaction files change continuation behavior, preserve summaries or retained tails, and guard against duplicated user messages or oversized tool results.	codex/codex-rs/core/src/session/turn.rs:345; goose/crates/goose/src/context_mgmt/mod.rs:1; opencode/packages/core/src/session/runner/llm.ts:145
E08	Product UX is part of the harness.	The cited UI, subscriber, delivery, and app files project runtime state to product surfaces rather than relying on terminal logs alone.	codex/codex-rs/tui/src/main.rs:50; goose/ui/desktop/src/App.tsx:598; opencode/packages/app/src/context/server-sdk.tsx:153
E09	Secret handling belongs below the agent interface.	The cited auth, OneCLI, secret-scope, and config files keep credentials behind keyrings, scoped maps, injected gateways, or config mediation rather than model-visible plaintext.	codex/codex-rs/core/src/config/auth_keyring.rs:10; nanoclaw/src/modules/approvals/onecli-approvals.ts:1; hermes-agent/agent/secret_scope.py:1
E10	Shell execution is a product decision, not a generic subprocess wrapper.	The cited shell paths encode cwd, timeouts, output budgets, sandbox or host selection, safe-bin or approval policy, and environment handling around subprocess execution.	codex/codex-rs/core/src/tools/handlers/unified_exec/exec_command.rs:80; goose/crates/goose/src/agents/platform_extensions/developer/shell.rs:351; opencode/packages/core/src/tool/bash.ts:113
E11	Agent systems need message delivery semantics, not just chat transcripts.	The cited routing and delivery files show that messages, files, reactions, edits, questions, and receipts require typed delivery semantics across product channels.	nanoclaw/src/router.ts:1; nanoclaw/src/delivery.ts:1; openclaw/src/channels/plugins/types.plugin.ts:54
E12	Repeated tool-call and malformed-tool recovery should be first-class.	The cited recovery files classify malformed calls, repeated calls, invalid JSON, tool-call text leakage, truncation, and provider/tool errors as specific harness states.	hermes-agent/agent/conversation_loop.py:3972; opencode/packages/opencode/src/session/processor.ts:330; openclaw/packages/llm-core/src/validation.ts:285
E13	A mature harness is observable at the run, tool, provider, and delivery layers.	The cited observability files emit lifecycle events, diagnostic states, telemetry spans, usage, failover, or sanitized observer data for production debugging.	codex/codex-rs/otel/src/provider.rs:77; openclaw/src/infra/diagnostic-events.ts:1; opencode/packages/core/src/observability.ts:10

ID	Claim	Observed behavior	Representative sources
E14	Test strategy follows harness boundaries.	The cited test and eval files show deterministic tests, live harnesses, LLM judge scripts, UI/e2e lanes, and CI decomposition around harness boundaries.	codex/codex-rs/core/tests/suite/otel.rs:113; goose/goose-self-test.yaml:66; goose/evals/harbor/runner.py:201
E15	A smaller core with a wider edge is the dominant shape.	The cited session/provider/plugin files keep model, tool, plugin, channel, and provider code behind contracts so additional surfaces reuse the same core loop.	codex/codex-rs/core/src/session/session.rs:51; goose/crates/goose/src/agents/agent.rs:236; opencode/packages/core/src/session/runner/model.ts:103
E16	File editing is a structured mutation workflow, not an arbitrary text rewrite.	The cited edit and patch files parse mutation intent, resolve paths, queue or guard writes, request edit approval, and render diffs instead of writing raw strings directly.	codex/codex-rs/core/src/tools/handlers/apply_patch.rs:1; goose/crates/goose/src/agents/platform_extensions/developer/edit.rs:1; opencode/packages/core/src/tool/edit.ts:1
E17	Browser and computer-use tools need their own lifecycle and audit boundary.	The cited browser and computer-control files isolate browser backends, expose accessibility/screenshot/CDP state intentionally, and test sandbox or bridge exposure.	goose/crates/goose-mcp/src/computercontroller/mod.rs:1; hermes-agent/tools/browser_tool.py:1; openclaw/src/agents/sandbox/browser.ts:1
E18	Subagent orchestration is strongest when child work has ownership, limits, permissions, and result re-entry.	The cited subagent files validate child-run parameters, depth and ownership metadata, permission inheritance, gateway/session dispatch, background delegate result re-entry, and, in NanoClaw, narrower long-lived agent creation through a host-authorized destination model.	openclaw/src/agents/subagent-spawn.ts:161; hermes-agent/run_agent.py:5231; opencode/packages/opencode/src/agent/subagent-permissions.ts:14
E19	Prompt architecture is rendered from layers with cache and hierarchy constraints.	The cited prompt files render layered system instructions, skill/context fragments, channel hints, and cache-stable sections through code-owned prompt builders.	codex/codex-rs/core/gpt_5_codex_prompt.md:1; goose/crates/goose/src/agents/prompt_manager.rs:11; opencode/packages/opencode/src/session/prompt.ts:16
E20	Evaluation and observability are separate but connected feedback loops.	The cited eval and observability files connect deterministic tests, live provider evals, judge scripts, prompt-cache diagnostics, and sanitized observer hooks into feedback loops.	goose/scripts/bench-postprocess-scripts/llm-judges/llm_judge.py:74; openclaw/src/agents/embedded-agent-runner/prompt-cache-observability.ts:158; hermes-agent/docs/observability/README.md:159

Appendix C: Glossary Of Harness Engineering Concepts

Term	Definition
Harness	The runtime around a model that owns context, tools, policy, persistence, state, and delivery.
Active run	The single in-flight unit of work draining a session's queued input and tool continuations.
Prompt admission	The step that durably records user input before the execution loop starts or continues.
Tool intent	A model-emitted request to use a capability, persisted before local side effects.
Settlement	The process of recording tool success, failure, interruption, provider execution, or denial in typed form.
Policy pipeline	Layered allow/deny/escalate decisions from profile, project, agent, group, sandbox, and dynamic approval sources.
Model line	The boundary above which the model sees rendered context and below which host authority and secrets remain controlled.
Context epoch	A versioned context baseline used to reason about prompt changes, compaction, and stale tool calls.
Compaction transition	A runtime step that replaces or summarizes history while preserving continuation semantics.
Delivery record	A typed outbound message, edit, file, reaction, question, or receipt owned by the harness.
Subagent	A child run or session with its own target, permissions, budget, transcript, and result channel.
Observer hook	A sanitized telemetry/event callback that lets plugins or operators inspect run behavior without raw object access.